CAPSTONE PROJECT



iEvaluate Batch Scanning System

Handover Guide

A plain-English guide for the Product Owner. Learn what the system is, how to run it, and what is still left to do.

DATE
2026-10-26

FOR
Product Owner

READING TIME
About 40 minutes

Before you start

This guide explains a big system in simple words. Every technical word is explained the first time it appears. All of them are also in the Glossary at the back.

THE STORY

1. What this system is
2. The whole system in one picture
3. How a form travels
4. Each part explained

DOING THINGS

5. What to install
6. Run the website on your PC
7. Install the folder watcher
8. Build the phone app
9. Look after n8n and the OMR server
10. The database and its changes
11. Everyday use

FIXING AND FINISHING

12. Troubleshooting
13. Known limits and future work
14. Pending items
15. Handover checklist

APPENDIX

- A. Settings lists
- B. Database migrations
- C. Review flag codes
- D. Glossary

TIP

Facts in this guide come from the code, the project notes and the n8n docs. Where something is not known, the guide says so and tells you how to check it.

WATCH OUT

No secret value is written in this guide. Secrets are shown only by name with the placeholder `<ask the previous developer>`. Ask the previous developer for the real values and keep them in a password manager.

MUST DO BEFORE HANDOVER

This colour means a job that must be finished before the handover is complete. Look for it in sections 9, 10, 14 and 15.

1 What this system is

Schools use a paper form called the **SAST**. Students rate their instructor on 20 questions by ticking a box. The Student Affairs Office (**SAO**) must turn these paper forms into scores. This system does that for them.

Path 1: Website

Staff scan a stack of forms on a scanner. The pages are sent on their own, read by software, scored and shown live. Forms that look wrong go to a **Needs review** list for a person to check.

Path 2: Phone

Staff take a photo of one form with the Android app called **iEvaluate**. It goes through the same reading and scoring.

Both paths save to the same cloud database. The phone app also lets instructors, department heads and admins see reports.

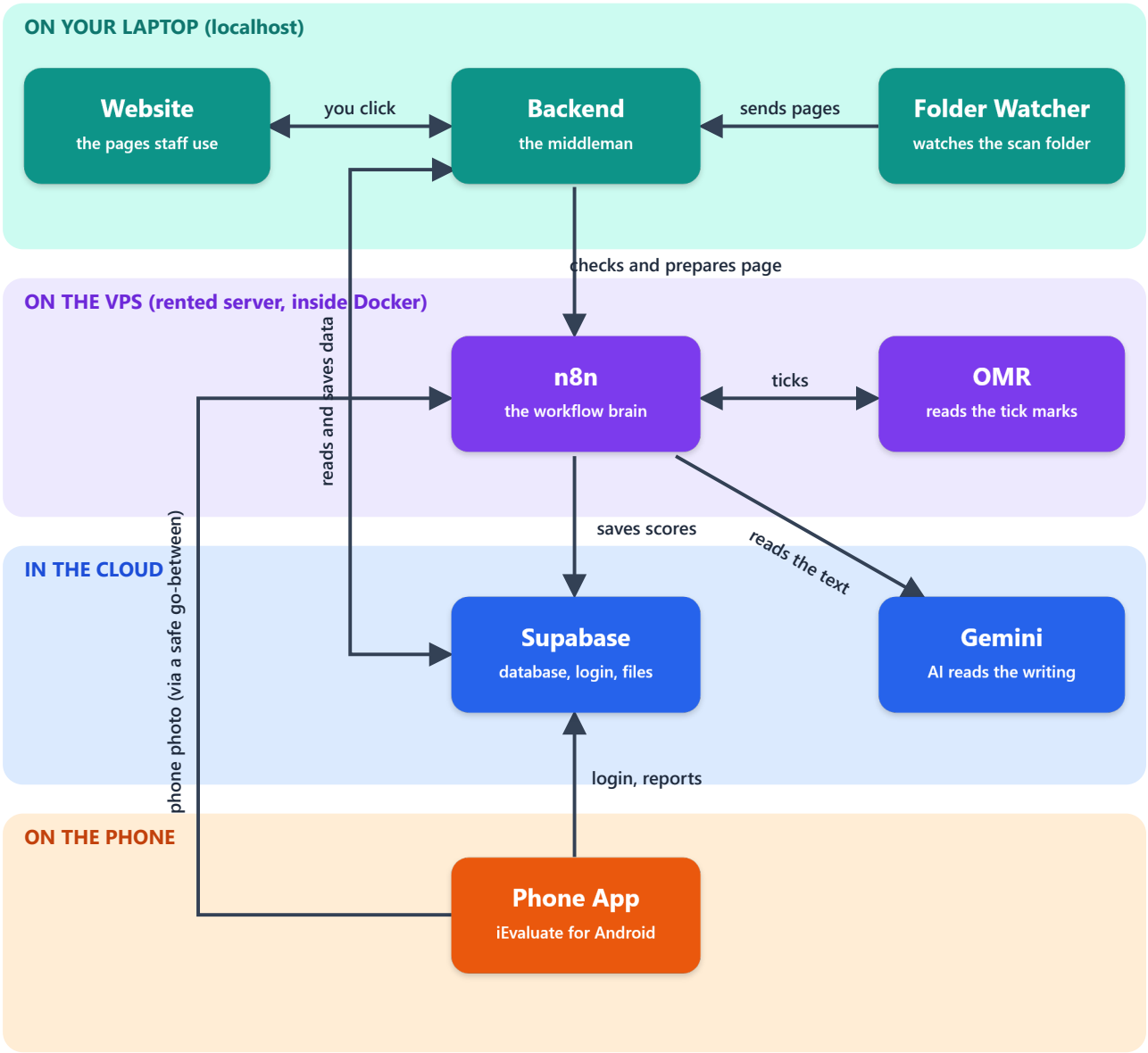
Who uses what

WHO	USES	WHAT THEY DO
SAO staff (`SAO_STAFF`)	Website, scanning PC, phone app	Scan forms and review problem forms
SAO admin (`SAO_ADMIN`)	Website, phone app	Everything staff do, plus activate instructor accounts and manage users, subjects, departments and terms
Instructor	Phone app only	See their own scores, remarks and AI suggestions
Department head or dean	Phone app only	See department results and write intervention reports (the app emails the instructor)

TIP

Instructors never log in to the website. Only approved accounts can log in (the account status must be `approved`).

2 The whole system in one picture



A paper form enters from the scanner or the phone. The folder watcher sends scans to the backend. The backend and the phone app both hand pages to n8n, which asks OMR and Gemini to read the form, then saves the scores to Supabase.

Where each part lives

PART	BUILT WITH	RUNS WHERE	FOLDER / LOCATION
Website screens	React 18, Vite 5	Your laptop, port 5173	`...\app\frontend`
Backend	Python, FastAPI	Your laptop, port 8000	`...\app\backend`
Folder watcher	Python 3.10+	Each scanning PC	`...\app\backend\watcher`
n8n workflow	n8n	VPS, Docker	`...\app\n8n\perfect-2.1.json`
OMR / OCR	Python, Flask, OpenCV	VPS, Docker, container `omr-api`	`C:\Projects\OMR PYTHON DEPLOYMENT` (separate repo)
AI	Google Gemini `gemini-3.5-flash-lite`	Google cloud, called by n8n	n/a
Database	Supabase (Postgres)	Cloud	SQL files in `...\app\migrations`
Phone app	Flutter / Dart	Android phones	`C:\Projects\ievaluateapp_final` (separate repo)
Mark labelling tool	Static web page	Any browser	`...\app\sast-mark-dataset\index.html`
OMR patch notes	Docs and test scripts	n/a	`...\app\omr_patch`

WATCH OUT

The website is **not deployed anywhere**. It only runs on a normal PC (localhost). That means the backend must run on the PC staff use, and the scanning PC must be able to reach it (see section 7). Hosting files for a future online launch exist (`render.yaml` , `frontend\vercel.json` , `backend\Dockerfile` , `backend\Procfile`) but are **not in use**.

3 How a form travels

Path 1: the website and scanner

1

Staff start a batch
Log in and open
Start
. A
batch
is one instructor + one subject + one scanning PC (a
station
). Only one batch can be open per station.

2

The scanner saves pages
Each page lands in the watched folder (default `C:\Scans`).

3

The watcher picks them up
It turns each file into a grey JPEG (about 200 dpi) and sends it, but only while a batch is open. Older files are moved to `skipped\before-batch`.

4

The backend checks the page
Valid station key? JPEG or PNG? 12 MB or less? Not sent already in this batch? Was the same image used in another batch (a "reused image")?

5

The backend tidies the page
It straightens it (upright, portrait), crops it to short bond (8.5 x 11) or long bond (8.5 x 13) and adds a thin grey border so the paper is easy to find.

6

The page goes to n8n
The backend posts it to n8n and waits up to 360 seconds.

7 **Two readers look at it**
n8n asks **OMR** which box is ticked on each of the 20 rows. It also asks **Gemini** to read printed and handwritten text (instructor, subject, semester, remarks, student ID, stamp number) and double-check the ticks.

8 **n8n compares and decides**
It compares both readings and matches the instructor, subject and student against the database.

9 **One of four answers**
success saved and counted. flagged sent to review with the reason. rejected not a SAST form, fewer than 5 of 20 answers, or a copied stamp. duplicate already seen, nothing changes.

10 **Scores are updated**
After a clean save, n8n recalculates the instructor averages, runs the AI remark analysis and may send a push alert (low score or critical remark).

11 **Staff review problem forms**
Open **Needs review**. Fix the values, then **Credit** (approve) or **Discard**. Approving calls n8n again.

12 **Staff close the batch**
A forgotten batch is closed by itself after 3 hours by default.

Path 2: the phone

1 **Take a photo**
In the app scanner. Check the result on the Data Validation screen.

2

The app calls a go-between

It calls the `n8n-proxy` function in Supabase. That checks the login and role, then forwards to n8n (`sast-scan-upload`). The app waits 60 seconds.

3

Same reading as before

n8n does the same OMR + Gemini reading. A phone has no batch, so the instructor and subject are read from the form text and matched in the database.

4

Save or fix

Clean forms are saved. Flagged forms go to a "failed scans" list in the app for a manual fix.

4 Each part explained

Each part has a colour tag. It matches the colour in the big picture.

1. Website

YOUR LAPTOP

WHAT IT DOES

The screens staff use: log in, start a batch, watch live counters, review problem forms. It talks only to the backend, never to Supabase or n8n directly.

WHERE IT LIVES

Folder `frontend`, port 5173.

HOW TO START IT

Run `npm run dev` inside the `frontend` folder (see section 6).

Key files: `App.jsx` (routes), `api.js` (calls to backend), `messages.js` (all wording). Login is kept in the browser session, so closing the browser logs you out. There are no frontend tests.

Website pages (staff only)

PAGE	ADDRESS	WHAT IT IS FOR
Login	`/login`	Staff sign in
Home	`/`	Getting-started checklist
Start a batch	`/start`	Pick or create instructor (placeholder if missing), subject, term
Live batch	`/batch/<id>`	Live counters, page list, page viewer, close batch
Review	`/batch/<id>/review`	Needs-review forms, import errors, "Couldn't be saved" pages. Fix, credit or discard
Review batches	`/review`	All batches the person may review
Scanner laptop	`/scanner-setup`	Online/Offline light, download watcher installer, change scan folder
Activation	`/activate`	Admin only. Give a placeholder instructor a real email so they can use the app (email sent through Brevo)
SAST guidelines	`/sast-guidelines`	Do's and don'ts for filling the form. Shown once per semester
Help	`/help`	"Something went wrong?" table

2. Folder Watcher (version 2.5.0)

YOUR SCANNING PC

WHAT IT DOES

A small Python program. It watches a folder, converts files and sends them to the backend. It proves who it is with a
STATION KEY
(a password for that PC).

WHERE IT LIVES

Each scanning PC. Source in `backend\watcher`.

HOW TO START IT

Double-click
START SCAN WATCHER
before scanning. Double-click
STOP SCAN WATCHER
after. It does
NOT
start by itself since version 2.5.0.

- Reads `.jpg .jpeg .png .tif .tiff .pdf`. Multi-page PDFs are split.
- Sends a **heartbeat** ("still alive") every 15 seconds. The website light turns red about 40 seconds after it stops.
- Sent files go to `<folder>\processed\<date>`. Pre-batch files go to `skipped\before-batch`. Unusable files go to `skipped`.
- Only one copy can run at a time. Network folders are refused.

3. Backend

YOUR LAPTOP

WHAT IT DOES

The middleman. It checks logins, owns batches and stations, receives pages, prepares images, calls n8n and stores page viewing copies.

WHERE IT LIVES

Folder `backend`, port 8000.

HOW TO START IT

Run `python -m uvicorn app.main:app --reload --port 8000` (see section 6). Check `http://localhost:8000/health`.

- Uses the Supabase **service key**, an all-powerful secret. It must never be in the browser or in git.
- Keeps page viewing copies (up to 1800 px) for 90 days in the private bucket `scan-pages`.
- Limits: 12 MB per page, 40 megapixels. On an error a page is tried up to 5 times, then marked failed. A timeout or network error is retried forever, so a dead n8n leaves pages "pending".
- Who sees what: admins see everything. Staff see their own batches and the batch of the laptop they are at.

Scoring and review rules

RULE	WHAT HAPPENS
Only a tick or slash counts	X, shading, a circle or a dot is treated as blank (not an error)
Blank is empty, not zero	Blank answers are left out of averages (migration 013)
Fewer than 5 of 20 answered	Auto-discarded and never counted. Staff cannot approve it (migration 014)
Python and Gemini disagree a lot	Flagged `omr_agreement_below_30`
Stamp number	Each form carries a numbering-machine stamp, unique per term. A copy is auto-discarded (`duplicate_stamp`). A missing or unreadable stamp goes to review (migration 015)
Reused image	The same image in another batch is flagged `reused_image`
Blank instructor, subject or student ID	Flagged
About 1 in 20 clean pages	Randomly sent to review as a spot check
Approve safety	You cannot overwrite another form of the same student without confirming
Sexual words in remarks	Masked with asterisks (insults are not masked)

4. n8n

VPS

WHAT IT DOES

N8N

is a tool where you connect steps ("nodes") on a canvas. Our workflow is the brain: it asks OMR and Gemini to read, compares them, decides and saves. Name: "perfect 2.1", 191 nodes, live id `vOMBjDtz15QB4k4u` .

WHERE IT LIVES

Rented Contabo server, in Docker. Workflow file: `n8n\perfect-2.1.json` .

HOW TO START IT

It is already running on the VPS. To update it, import the file (section 9).

- Alerts are push notifications (Firebase). n8n sends **no emails**. Activation email comes from the backend (Brevo). Intervention emails come from a Supabase function used by the phone app.
- Source pieces live in `n8n\tools\website_src` . Main technical doc: `n8n\WEBSITE_BRANCH.md` .

WATCH OUT

Do not rearrange the canvas or set an Error Workflow in settings. The order of branches matters. Node names contain typos (`subjetc` , `departmeht`). Keep them, because code refers to nodes by name.

The n8n doors (webhooks)

A **webhook** is a web address that starts the workflow when something is sent to it.

WEBHOOK	CALLED BY	PURPOSE
`sast-batch-scan`	Website backend (needs `X-Webhook-Secret` header)	One scanned page
`sast-scan-upload`	Phone app	One phone photo
`sast-manual-correction`	Website Review page, phone app	Staff fixes a flagged scan and credits it
`import-error-correction`	Phone app, backend	Fix a Google Sheet import row
`sast-link-upload`	Phone app	Import a Google Sheet of results
`subject-bulk-import`	Phone app	Bulk create subjects
`health-check`	Phone app	Returns "online"

5. OMR / OCR service

VPS

WHAT IT DOES

OMR

= reading ticked boxes.

OCR

= reading text. This service is classic computer vision, not an AI service. It finds the paper and the 20-row table, decides which column is ticked in each row (only ticks count), and reads the stamp number with its own small digit reader.

WHERE IT LIVES

VPS, Docker container `omr-api` (port 5000, inside the Docker network only). Code: `C:\Projects\OMR PYTHON DEPLOYMENT` (own git repo).

HOW TO START IT

Restart: `docker compose restart omr-api` on the VPS (section 9).

- Gemini's stamp read must match the OMR digits exactly, otherwise the form goes to review.
- Header text OCR (EasyOCR) is switched **off** (`OCR_MODE=off`) because it was about 0% accurate on handwriting. Gemini reads the header instead.
- Endpoints: `POST /process` (form image in, ratings and stamp out), `POST /process-crop` (one hand-drawn crop from the phone), `GET /health` .
- Accuracy (small tests): about 97% of rows on tuning sets; 37% to 98.8% on the new table print after the 2026-10-08 fix. It is weak when the photo shows only part of the page or is rotated. It then silently guesses a fixed grid (flag `omr_fallback_grid`).

6. Gemini AI

GOOGLE CLOUD

WHAT IT DOES

Google's AI model reads the printed and handwritten text on the form and double-checks the ticks. Model: `gemini-3.5-flash-lite`, called directly from n8n (not through OpenRouter). It also analyses remarks (themes, word cloud, tone, summary).

WHERE IT LIVES

Google's cloud. Called by n8n with an API key stored as an n8n credential.

HOW TO START IT

Nothing to start. Make sure the n8n credential holds a working key.

- **What Gemini gets:** the form image (student ID and signature are visible in it), the list of known SUBJECTS, and instructions.
- **What Gemini never gets: instructor names.** This is a school rule. Instructors are matched in code against the database. The build and tests check this. Trade-off: badly garbled names go to staff review.
- Free keys hit a request limit (error 429). The key is shared with production n8n. Whether it is on a paid tier is unknown.

WATCH OUT

Privacy gap. The capstone paper claims OpenRouter, "anonymization" and student-ID hashing. The code does none of these. Student IDs are stored in plain text, and Google is not named as a data processor in the app's privacy notice. Either the paper or the code must be corrected (section 14).

7. Supabase

CLOUD

WHAT IT DOES

SUPABASE

is a hosted database (Postgres) with login and file storage. The website and phone app share one project. The project address is hard-coded in 8 n8n nodes.

WHERE IT LIVES

Cloud. SQL change files are in `...\app\migrations`.

HOW TO START IT

Nothing to start. Database changes are applied by hand in the Supabase SQL Editor (section 10 and Appendix B).

- Main tables include `user_info`, `Sao_users`, `stations`, `scan_batches` (the raw scores), `sast_all_raw_data_survey`, `student_remarks` and about 30 more.
- Key functions: `recompute_sast(instructor, term)` (rebuilds averages), `save_website_raw_row`, `sast_stamp_duplicate_of`.
- Private storage bucket `scan-pages` holds page viewing copies.
- **Edge functions** (small programs inside Supabase, kept in the Flutter repo): `n8n-proxy`, `send-intervention-email`, `update-system-settings` and admin ones.
- **RLS** (row-level security) limits who can read each table. Migration 007 fixed a self-signup privilege hole.

8. Phone App (iEvaluate)

ANDROID PHONE

WHAT IT DOES

The Android app for scanning single forms and for viewing reports. Version 1.0.0+1. App id `com.example.ievaluateapp_final` (a default name; change it before any store release).

WHERE IT LIVES

Repo `C:\Projects\ievaluateapp_final` (separate from the website).

HOW TO START IT

Build an APK and install it on the phone (section 8).

It talks to Supabase directly and to n8n only through `n8n-proxy`. Push messages use Firebase Cloud Messaging.

What each person sees in the app

ROLE IN APP	DATABASE ROLE	SCREENS
Gatherer (SAO staff)	`SAO_STAFF`	Dashboard, camera scanner, data validation, sync queue, failed scans, Google Sheet import, settings
Instructor	FULL-TIME / PART-TIME / INSTRUCTOR	Dashboard, subjects, student feedback, past terms, PDF report
Dept head / dean	DEPARTMENT-HEAD / DEAN	Overview, faculty roster, subject analytics, intervention reports
SAO admin	`SAO_ADMIN`	User, personnel, subject and department management, system audit, import errors, live metrics, settings

5 What to install

NEEDED FOR	SOFTWARE	VERSION	NOTES
Website backend	Python	3.11 or 3.12	Docker image uses 3.11. Check with `python --version`
Website frontend	Node.js + npm	Use 20 LTS (not recorded)	Vite 5 / React 18 need Node 18+ (assumption). Check with `node --version`
All	Git	Any recent	
Watcher PC	Python	3.10 or newer (3.10 to 3.14 work)	The installer can install it for you (winget)
OMR (local test only)	Python	3.10	Matches the production container
n8n + OMR on VPS	Docker + Docker Compose	On the VPS	Already installed there
Phone app	Flutter	3.41.6 stable (Dart 3.11.4)	Used by the previous developer
Phone app	Android Studio (Android SDK)	Any recent	
Phone app	Supabase CLI	Any recent	Only to deploy edge functions
n8n tools (optional)	Python 3 and Node	Any	For build and test scripts

Accounts you need

Supabase project · Google (Gemini key) · Google service accounts (Firebase, Sheets) · Brevo (email) · GitHub repos · Contabo VPS · DuckDNS (name of the n8n address)

6 Run the website on your PC

Start from a fresh Windows PC. You need about 20 minutes.

1 Install the tools

Install Python, Node.js and Git (section 5). Open a **new** terminal afterwards.

2 Get the code

Clone or copy the `app` folder. Open a terminal in it.

3 Get the database ready

Apply the migrations (section 10). Have the Supabase project URL and the **service key** ready.

4 Create the backend settings file

Copy `backend\.env.example` to `backend\.env`. Fill in `SUPABASE_URL` and `SUPABASE_SERVICE_KEY` (secret). For real scoring also set `N8N_BATCH_WEBHOOK_URL` and `N8N_WEBHOOK_SECRET` (secret, must equal the value in the n8n credential). The backend refuses to start if the URL is set without the secret.

For a demo without n8n, leave the URL empty and set `SIMULATE_SCANS=true`. You get fake results.

> **WATCH OUT**
>
> `.env` is private. Never commit it or email it.

5 Install and start the backend

```
cd backend
python -m venv venv
venv\Scripts\python -m pip install -r requirements.txt
venv\Scripts\python -m uvicorn app.main:app --reload --port 8000
```

6 Check it works

Open `http://localhost:8000/health`. You should see `{"status":"ok"}`.

7 **Start the frontend in a second terminal**

```
cd frontend
npm install
npm run dev
```

8 **Open the website**

Go to `http://localhost:5173`. The frontend finds the backend at `http://localhost:8000`. To change that, set `VITE_API_BASE`.

9 **Log in**

You need an approved account: a row in `user_info` with `account_status = approved` and a row in `Sao_users` with role `SAO_STAFF` or `SAO_ADMIN`. A new station is registered the first time you use the "Scanner laptop" page.

> **TIP**
>
> Create the very first admin like this: add the person in Supabase Auth, then a `user_info` row (`account_status = approved`) and a `Sao_users` row (role `SAO_ADMIN`).

Run the tests

1 **Backend tests**

```
cd backend
venv\Scripts\python -m pip install -r requirements-dev.txt
venv\Scripts\python -m pytest -q
```

Tests do not write to the database. One test (`test_orientation` , real samples) failed before the recent changes and may still fail.

2 n8n tests (from the n8n folder)

COMMAND	CHECKS
`python tools/validate_workflow.py`	Workflow structure and rules (optional `pip install pglast`)
`node tools/test_website_code.js`	The workflow's code nodes
`node tools/test_website_sql.js`	Its SQL. Needs a throwaway local Postgres. Refuses non-local hosts

3 Phone app tests

```
flutter test
```

7 Install the folder watcher on a scanning PC

> **WATCH OUT**
>
> The website runs only on localhost, so the watcher can reach it only if the backend is reachable from the scanning PC. Easiest: run the backend and website on the scanning PC itself, or on a PC in the same network with `WATCHER_API_BASE` and `PUBLIC_API_BASE` set to its address. The right address for your setup is not recorded. Confirm with the previous developer.

Staff can do these steps. No technical skill is needed.

1 **Open Scanner setup**

Log in to the website on that PC and open **Scanner setup**.

2 **Choose the station**

Choose or create the station (the laptop's name).

3 **Download the installer**

Click **Download installer**. You get `ScanWatcher-<station>.zip`.

> **WATCH OUT**
>
> Each download makes a **new station key**, so the old PC using that station stops working.

4 **Extract the zip**

Right-click the zip and choose **Extract All**.

5 **Run the installer**

Open the extracted `ScanWatcher` folder and double-click `INSTALL.bat`.

6 **Wait for ALL DONE**

Wait for the green "ALL DONE" window. The installer finds or installs Python, installs what it needs, copies the program to `%LOCALAPPDATA%\ScanWatcher`, creates `C:\Scans` and adds **Start Scan Watcher** and **Stop Scan Watcher** shortcuts.

7 **Point the scanner at the folder**

In the scanner software, set the save folder to `C:\Scans`.

8 **Start it before each session**

Double-click **Start Scan Watcher**. The website light should turn green (Online).

Good to know

- If the installer window closes at once, the zip was not extracted.
- To remove it run `UNINSTALL.bat`. To check status run `CHECK.bat`.
- IT can make it start at log-on with `install_task.bat` (undo with `uninstall_task.bat`). Running `INSTALL.bat` again turns auto-start off.
- Laptops installed before 2.5.0 have no shortcuts and must reinstall. The website shows a note when the version is older.
- Files in `backend\watcher`: `INSTALL.bat`, `UNINSTALL.bat`, `CHECK.bat`, `STOP.bat`, `shortcuts.bat`, `start_watcher.bat`, `watcher_process.bat`, `find_python.bat`, `setup_helper.py`, `folder_watcher.py`, `build_package.py`.
- Guides: `HOW TO INSTALL.txt` (end users), `TECHNICAL NOTES.txt` (IT), `INSTALL_STEPS.md` (wording shown on the website; keep it in sync).

Changing the watcher program

The installer zip is `backend\app\static\watcher-package.zip`. After **any** watcher edit, rebuild it:

```
cd backend
venv\Scripts\python watcher\build_package.py
```

> **WATCH OUT**
>
> Never put a `watcher_config.json` inside the zip.

8 Build the phone app

Do this on the developer PC, in `C:\Projects\ievaluateapp_final`.

1 Install the tools

Install Flutter 3.41.6 and Android Studio. Run `flutter doctor` and fix what it reports.

2 Fetch packages

```
flutter pub get
```

3 Create the settings file

Create `.env.json` in the project root (never commit it). Key names: `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `N8N_BASE_URL`, `USE_N8N_TEST`, `N8N_WEBHOOK_HEALTH`, `N8N_WEBHOOK_SCAN_UPLOAD`, `N8N_WEBHOOK_LINK_UPLOAD`, `N8N_WEBHOOK_CROP_OCR`, `N8N_WEBHOOK_MANUAL_CORRECTION`, `N8N_WEBHOOK_SUBJECT_BULK_IMPORT`, `N8N_WEBHOOK_IMPORT_ERROR_CORRECTION`, `PYTHON_PROCESS_CROP`. Optional: `USE_DIRECT_N8N`, `SHEET_SERVICE_ACCOUNT_EMAIL`. Values: `<ask the previous developer>`.

4 Test run on a connected phone

```
flutter run --dart-define-from-file=.env.json
```

5 Build the release file

```
flutter build apk --release --dart-define-from-file=.env.json
```

> **WATCH OUT**
>
> Without the define file, login fails (the address and key are empty).

6 Find the file

```
build\app\outputs\flutter-apk\app-release.apk
```

7

Install it on a phone

Turn on USB debugging and run the command below, or copy the APK to the phone and open it.

```
adb install -r build\app\outputs\flutter-apk\app-release.apk
```

Important things about the phone app

MUST DO BEFORE HANDOVER

Back up the signing key. Release builds use `android\app\keystore\release.jks`. If you lose it, users must uninstall before they can update. If the file is missing, the build silently uses a debug key. The alias and password have default values written inside `build.gradle.kts`. Remove and rotate them. Environment overrides: `KEYSTORE_FILE`, `KEYSTORE_PASSWORD`, `KEY_ALIAS`, `KEY_PASSWORD`.

WATCH OUT

Debug and release builds have different signatures. Switching gives `INSTALL_FAILED_UPDATE_INCOMPATIBLE`. Fix: `adb uninstall com.example.ievaluateapp_final`. This **deletes scans not yet synced**.

TIP

Judge speed on release builds only. Debug builds lag. The Android `INTERNET` permission is in `AndroidManifest.xml` (without it, release login broke).

Deploy a Supabase function

```
supabase functions deploy <name>
```

9 Look after n8n and the OMR server

WATCH OUT

The previous developer did not log in to the server while writing this guide. The values below come from project notes. Check them on the server.

The server

- Provider: Contabo VPS. Stack folder: `/root/ievaluate-stack/` (Docker Compose). Login: `<ask the previous developer>`.
- The n8n address is a DuckDNS name served over HTTPS by **Caddy** (`docker-compose.override.yml` + `Caddyfile` ; backup `docker-compose.yml.bak`). Exact name: `<ask the previous developer>`.
- Three containers: n8n, `omr-api` (port 5000, inside Docker only; n8n reaches it as `http://omr-api:5000/process`) and Caddy.

MUST DO BEFORE HANDOVER

Not yet done on the VPS: close the old open port 5678, set the Supabase secret `N8N_BASE_URL` for `n8n-proxy` , and reboot the server (it asked for a restart).

Import the workflow into n8n

1 **Open the workflow**

Open n8n and open the workflow "perfect 2.1".

2 **Import**

Top-right menu, **Import from File**, choose `n8n\perfect-2.1.json` . This replaces the canvas and keeps the web addresses.

3 **Fix credential markers**

Any node with a red credential marker: pick the right credential from its dropdown.

4 **Save and activate**

Save. Switch the workflow to **Active**. (The docs say the file imports inactive, but the file in the repo has `active: true` . Check after import.) Webhook addresses starting `/webhook/` only work while Active.

5 **Check the request size**

If the default 16 MB request size is too small, set `N8N_PAYLOAD_SIZE_MAX`. Pages are about 1 MB.

Credentials to create in n8n

The names must match exactly. All values: `<ask the previous developer>`. After a Supabase key change, update both n8n and the backend `.env`.

NAME	TYPE	VALUE
`iEvaluate website webhook secret`	Header Auth, header `X-Webhook-Secret`	Same as backend `N8N_WEBHOOK_SECRET`
`Supabase iEvaluate (secret key)`	Supabase API (host = project URL)	Supabase service key
`Postgres account`	Postgres	Supabase database connection
`Google Gemini(PaLM) Api account`	Google Gemini (PaLM) API	Gemini key
`firebase pushnotification`	Google API (service account)	Push messages, Firebase project `ievaluate-pushnotification`
`Google Service Account account 2`	Google API (service account)	Google Sheets

Restart or update the OMR service

1 **Log in**

Log in to the VPS and go to `/root/ievaluate-stack/`.

2 **Restart**

```
docker compose restart omr-api
```

3 **Or update**

Copy the changed files into the stack's `omr-api` folder, then run:

```
docker compose up -d --build omr-api
```

4 **Check it**

```
curl http://localhost:5000/health
```

Run it from inside the Docker network. It can refuse for 30 to 60 seconds after a restart.

5 **Compare files**

After every update, compare file checksums of `api.py`, `mark_detect.py`, `grid_detect.py`, `utlis.py`, `grid_config.py` and the stamp files with your local copies. A past rollback left old files running.

6 **Know the rollback aids**

Backups under `/root/omr_backup/<date>`, `/root/omr-api.bak-<date>` and the Docker image tag `omr-api:rollback`.

7 **Check the phone photo limits**

`OMR_MAX_LONG_SIDE` and `OMR_CAP_LONG_SIDE` should be **4200** and **3264**. The local `api.py` still shows 3000 and 1920; the notes say the VPS runs the higher values. Confirm on the VPS.

Change the workflow

TIP

Avoid editing the JSON by hand. Edit the small source pieces instead.

1 **Edit the pieces**

Edit files in `n8n\tools\website_src`.

2 **Build from a fresh live export**

From the `n8n` folder:

```
python tools/build_website_branch.py --live "<exported live workflow>.json"
../perfect-2.1.json
```

3 **Run the tests**

See section 6.

4

Import into n8n

Follow "Import the workflow into n8n" above.

WATCH OUT

The build script was written to read input files from the previous developer's Downloads folder (`perfect 101 (4).json`). They may be gone. Always start from a fresh export of the live workflow.

10 The database and its changes

A **migration** is a SQL file that changes the database. There is **no automatic runner** and no "applied" log. Each file is run by hand in the Supabase SQL Editor. Read each one first. Most can be safely run again.

> **TIP**
>
> To see what is applied, ask a developer to run Supabase `list_migrations`, or look for the table or column a migration creates. The full list with a "How to check" column is in Appendix B.

> **WATCH OUT**
>
> Two files share number 004 and two share 007. Run them by name. The README list is out of date (it omits 006 to 018). The phone repo has its own migrations in `supabase\migrations` (latest `20240130000033`).

Extra checks

- The unique index on `sast_all_raw_data_survey` must be a **plain** unique index, not `NULLS NOT DISTINCT`. Otherwise forms with a blank student ID after the first are flagged `duplicate_student_id`. The check SQL is in `n8n\WEBSITE_BRANCH.md`, section "Unique key". Not yet checked on the live database.
- The database needs the `pg_trgm` extension (name similarity).

> **MUST DO BEFORE HANDOVER**
>
> Migration `015b_delete_dummy_scan_data` is **destructive**. It deletes test scans (it backs up to `backup_*` tables). Run it only on purpose.

Safe order for an update

1. Apply the migration → **2. Import the n8n workflow** → **3. Build the new phone app**

11 Everyday use

Staff: each scanning session

1 Start the watcher

Double-click **Start Scan Watcher** on the scanning PC.

2 Check the light

Log in to the website. On **Scanner laptop**, the light is green.

3 Start a batch

Click **Start**. Choose the instructor, subject and term.

4 Scan the forms

Scan into `C:\Scans`. Watch the counters on the live page.

5 Finish

When the pile is done, close the batch. Double-click **Stop Scan Watcher**.

6 Review

Open **Needs a look**. For each form compare the picture with the typed values, fix, then **Credit** or **Discard**. For stamp problems type the stamp number, or choose "Form has no stamp".

Admin

- **Activate an instructor:** open **Activation**. Type the real email twice. The instructor gets an email.
- Manage users, subjects, departments and terms in the phone app.
- Each scanning laptop is a "station". Regenerating its key stops the old PC.

> **TIP**
>
> Approval states: "Sending..." then "Not sent" after 10 minutes if n8n does not answer.

12 Troubleshooting

Find your problem on the left. Do the fix on the right.

PROBLEM	FIX
Website light is red (Offline)	→ Watcher not running. Double-click **Start Scan Watcher**. Run `CHECK.bat`.
"Folder not found" or "That folder isn't allowed"	→ Wrong or network folder. Set the folder to `C:\Scans` on the Scanner setup page.
Installer window closes at once	→ The zip was not extracted. Right-click it, **Extract All**, run again.
Scans do not appear	→ Scanner saves elsewhere, or no open batch. Point the scanner at `C:\Scans`. Open a batch first.
Old files not sent	→ They are older than the batch. Find them in `skipped\before-batch` and rescan.
An old laptop stopped after another download	→ The download made a new station key. Reinstall on the right PC.
Pages stay "Working on it"	→ n8n or OMR is down or slow (timeouts retry forever). Check n8n is Active and `omr-api` is running (section 9). Pages continue afterwards.
Everything goes to review as `stamp_unconfirmed`	→ The OMR service is not sending stamp data. Deploy the current OMR files and check `STAMP_FINDER` and `STAMP_READER` are `on`.
Many `omr_fallback_grid` or weak rows on phone photos	→ Image size cap too low on the OMR server. See section 9, "Check the phone photo limits".
Backend says `schema_outdated` or `migration_required`	→ Migration 002 or 005 is not applied. Apply it (section 10).
Backend will not start	→ `N8N_BATCH_WEBHOOK_URL` is set without `N8N_WEBHOOK_SECRET`, or Supabase keys are missing. Fix `backend\.env`.
n8n answers 401 or 403, backend shows 503	→ Webhook secret mismatch. Make the n8n credential equal `N8N_WEBHOOK_SECRET`.
Flagged pages fail to save (401)	→ The n8n credential `Supabase iEvaluate (secret key)` has an old key. Paste the current key.
Sheet import fails with 403	→ The sheet is not shared. Share it with the service account email (Viewer).
Phone app login fails on a release build	→ Built without the define file. Rebuild with `--dart-define-from-file=.env.json`.
`INSTALL_FAILED_UPDATE_INCOMPATIBLE`	→ Debug and release signatures differ. Run `adb uninstall com.example.ievaluateapp_final`, then install.

PROBLEM	FIX
Gemini 429 errors	→ Request limit. Wait, or use a paid key.
Scores look different after a fix	→ Expected after migration 013: averages are recalculated and blanks no longer count as zero.
Review shows "Already approved / changed" (409)	→ Someone else edited the row. Reload and look again.
Browser logs out	→ Login is kept for the session only. Log in again.
A page shows no preview	→ Migration 010 or the bucket is missing. Apply 010 and create the bucket `scan-pages`.

13 Known limits and future work

What the system cannot do well today

- The website is localhost only. No online deployment.
- Sessions end when the browser closes. Login attempts are rate-limited per IP.
- Only a tick or slash counts as an answer. This is a rule-based check, not a trained model.
- **Table finding is fragile.** A photo where the page touches the frame, is cut off, rotated, or shows only the table can fail silently and use a guessed grid.
- Dots versus tiny real ticks cannot be told apart reliably. Pencil shading and circles are read less well.
- Stamp numbers of 5 or 6 digits never auto-accept. Auto-accept was 17 of 29 pilot pages with 0 wrong.
- Gemini is the weaker reader of ticks, so Python wins when they disagree. Thresholds were not re-measured after the latest rules.
- Garbled instructor names go to staff review (no AI help, by design).
- Re-importing the same Google Sheet adds blank-student rows again. A fix idea (a stable `task_id` per row) is not built.
- An n8n crash during a sheet import is not caught (the app stops waiting after 60 minutes).
- Google Sheet import errors show only in the phone app, not on the website.
- No frontend tests. Backend tests do not touch the database.

Measured: how often the table was found

PHOTO TYPE	FOUND CORRECTLY
Normal framing	10 of 10
Page touching the frame	1 of 10
Cut-off edges	2 of 10
Table only	0 of 10
Rotated 5 degrees	3 of 10

Website audit items not yet decided

- n8n down leaves pages "Working on it" forever (no error mark).
- A re-sent file shows no "Already scanned".
- The forgotten-batch reason is only a tooltip.
- Laptop names are case-sensitive.
- Weak activation-email check.
- Student-ID format is not normalised for the overwrite guard.

- The backend still sends 0 for a blank in `review_logic.py` (around lines 630 and 653). n8n converts it, but it should be `None`.

Phone app tech debt and open security items

- Gatherer screens bypass the main scaffold; default app id; keystore passwords in gradle; README mentions a deleted screen.
- The `user_info` policy "Allow anonymous email lookup" lets anonymous users read all profiles (needs an app change).
- `admin-accept-user` does not check the caller is approved.
- HTML escaping in `update-system-settings`.
- The `intervention_reports` insert rule needs tightening.
- One account holds both SAO_STAFF and FULL-TIME roles. Check it by hand.

Future work

> **WORK IS FROZEN**
>
> The owner froze all Python OCR/OMR work until after the capstone presentation. The ideas below are for a future student or professor.

- **Better table finding:** search at several scales, try all 4 rotations, match the form's own line pattern, send "table not found" to review instead of guessing, add a learned corner detector. Printed corner markers would be ideal, but the school cannot reprint the form.
- **Data needed:** 30 to 50 hard phone photos with corners marked, and 3 to 5 blank 300 dpi forms. Use the labelling tool `sast-mark-dataset\index.html`. Zips go to `C:\Projects\OMR PYTHON DEPLOYMENT\dataset`. This data holds real names and IDs, so keep it out of git.
- A check-only mark classifier trained on that dataset (replaces the rule-based check).
- Header OCR training (printable sheets exist in `tools\handwriting_sheets`), a student-ID ink cross-check, extra stamp reader training.
- The teacher's list of valid stamp prefixes and lengths (currently a placeholder).
- Deploy online (Render + Vercel files exist) and the fixes in the pre-deploy audit.
- Phone text and screens for the newer review reasons (`stamp_unconfirmed` and others).

14 Pending items

These come from dated project notes. Check the live state before acting, because some may already be done.

MUST DO BEFORE HANDOVER

Rotate these now.

1. The Supabase secret key. An old one was found inside an n8n export. Rotation is still pending (`SETUP.md` step 0). After rotating, update the backend `.env` and the n8n credential `Supabase iEvaluate (secret key)`.
2. The n8n API key (it was shared in chat).
3. The hard-coded keystore alias and passwords in `build.gradle.kts`.
4. Delete old `perfect 101*.json` exports from the previous developer's Downloads folder (they may contain the old key).

Not committed (work only on the PC)

1 Website repo

Watcher 2.5.0, review and stamp changes, frontend changes, `n8n\perfect-2.1.json`, `SastGuide.jsx`, tests, `watcher-package.zip` (many files).

2 OMR repo

`api.py`, `utlis.py`, `requirements.txt`, `Dockerfile`, `.dockerignore`, `stamp_find.py`, `stamp_read.py`, weights and templates, `tools`. Most are already on the VPS. **Do not commit** `dataset` or `eval_out` (real names and IDs).

3 Phone repo

Changes to `env.dart`, `failed_scan_detail_screen.dart` (large), `gatherer_settings_view.dart`, `admin_dashboard.dart`, `sao_admin_settings.dart` and others; new `correction_pickers.dart`, `sheet_import_job.dart`, `lib\sast_guide` and 4 tests. A `.mcp.json` file must **not** be committed.

4 Unpushed commits

Branch `reused-image-and-heartbeat` (commit 76784c7) was never pushed. The website `main` branch has later commits (68bcea4, 0906f7b) also not pushed.

Not deployed or not applied

5 **n8n workflow**

Re-import `n8n\perfect-2.1.json` (reused-image check, no instructor names to Gemini on the phone path, stamp rule, id-based corrections, min-5 rule). An import happened about 2026-10-08. Whether the newest version is live is unknown.

6 **Migrations**

013 and 014 were reported live (2026-10-08 and 10-16). 012, 015, 016 and 018 (and 009, 010, 011, 017) were not confirmed. 015b is destructive. For 015, do not turn the stamp rule on until the OMR stamp files, the n8n stamp step and the teacher's prefix list are settled.

7 **Website backend and frontend**

Restart the backend and rebuild the frontend after the min-5 and review changes. Unverified.

8 **Phone app**

Build a new APK **after** migration 013 and the n8n import, in that order. Also confirm app averages match the database after 013 (a shared `sast_rating` util was added).

9 **Phone text for new reason codes**

`reused_image`, `instructor_missing`, `subject_missing`, `student_id_missing`, `stamp_unconfirmed`.

10 **Supabase function `send-intervention-email`**

It was open to anyone and injectable. The fix (login check, dean check, `report_id`, HTML escaping, `verify_jwt=true`) is probably committed (commit 231242e). Confirm it is deployed with `supabase functions deploy send-intervention-email` and the `emailed_at` migration is applied. Follow-ups: tighten the `intervention_reports` insert rule, escape HTML in `update-system-settings`.

11 **Loose OMR files**

In the previous developer's scratch folders, not in the repo, not deployed: the v2 table finder (frozen) and the dot rule (`OMR_DOT_MIN_FRAC`). Decide with the future team.

12 **Watcher 2.5.0**

Every scanning laptop must re-run `INSTALL.bat`.

13 **VPS clean-up**

Close port 5678, set the `N8N_BASE_URL` secret for `n8n-proxy`, reboot the server.

14 **Check one phone scan**

Confirm `omr_input_scale` is 1 and `grid_source` is `structure-fit`.

15 **Check the unique index**

Confirm the unique index on `sast_all_raw_data_survey` is not `NULLS NOT DISTINCT`.

16 **Tidy up**

Delete the dead file `n8n\tools\website_src\recompute_aggregates.sql` if it still exists. Delete the test placeholder instructor "rodz Licayan" and reset test laptops' folder to `C:\Scans`.

17 **Audit bugs**

Website audit bugs awaiting a decision (section 13).

18 **Privacy paperwork**

- Add Google / Gemini as a named processor in the signed privacy notice (`lib\core\config\agreements.dart` in the phone repo).
- Either build student-ID hashing or remove the claim.
- Correct the paper: no OpenRouter, no anonymization, model is `gemini-3.5-flash-lite`.
- Check whether the Gemini key is on a paid tier (it affects whether Google may train on the data).

19 **Teacher's answer needed**

The valid stamp prefix and length list.

20 **Team rule**

The previous developer never pushed or deployed for the owner. Use the commands in this guide yourself.

15 Handover checklist

What to give the Product Owner

ITEM	WHERE / NOTE
Website repo	`c:\Projects\shisss complee new website\app` (git, branch main). Many uncommitted changes (section 14)
Phone app repo	`C:\Projects\ievaluateapp_final`
OMR repo	`C:\Projects\OMR PYTHON DEPLOYMENT` (GitHub 4harvey7/OMR_Python_for_SAST). Has uncommitted files already running on the VPS
Documents	README.md, SETUP.md, n8n\WEBSITE_BRANCH.md, n8n\README, omr_patch\README.md, watcher guides, phone README.md and MANUAL.md, docs\UAT_*.txt, UAT-iEvaluate-Batch-Scanning-Website.docx
Accounts	Supabase (owner), Google (Gemini key, Firebase, Sheets service accounts), Brevo, GitHub, Contabo VPS, DuckDNS, Android keystore owner
Server	VPS login and Docker stack access: `<ask the previous developer>`

Secrets to hand over safely

MUST DO BEFORE HANDOVER

Hand these over through a **password manager**, never by email or chat, and rotate them. Values: `<ask the previous developer>`.

SECRET	USED IN
`SUPABASE_SERVICE_KEY`	Backend .env, n8n credential
Supabase database password	n8n `Postgres account`
`SUPABASE_ANON_KEY`	Phone app .env.json
`N8N_WEBHOOK_SECRET`	Backend .env, n8n credential
n8n admin login and n8n API key	n8n
Gemini API key	n8n credential
Firebase and Google service-account files	n8n credentials, `google-services.json`
`BREVO_API_KEY`	Backend .env
Station keys	One per laptop. Regenerate when handing over
`release.jks` and its passwords	Phone app signing. Back it up
VPS login	Server

First-week checklist for the new owner

- [ ] Receive repos, accounts and secrets through a password manager
- [ ] Rotate the keys listed in section 14
- [ ] Run the website on localhost (section 6) and run the tests
- [ ] Log in to the VPS. Confirm n8n and `omr-api` are running
- [ ] Check which migrations are applied (section 10 and Appendix B)
- [ ] Import the latest workflow and check credentials (section 9)
- [ ] Commit the uncommitted work after review
- [ ] Reinstall the watcher on each scanning PC (section 7)
- [ ] Build and test the phone APK (section 8)

- [ ] Scan 5 test forms end to end. Confirm one reaches review and one is credited

Before you say "done"

- [ ] Supabase secret key rotated, backend .env and n8n credential updated
- [ ] n8n API key rotated
- [ ] Keystore passwords removed from build.gradle.kts and rotated; `release.jks` backed up
- [ ] Old `perfect 101*.json` exports deleted
- [ ] VPS: port 5678 closed, `N8N_BASE_URL` secret set, server rebooted
- [ ] Privacy paper and privacy notice agree with the code
- [ ] Teacher's stamp prefix list received

APPENDIX

A Settings lists

Backend settings (names only)

Defaults are in `backend\.env.example`.

NAME	MEANING
`SUPABASE_URL`, `SUPABASE_SERVICE_KEY`	Database address and secret key (required)
`N8N_BATCH_WEBHOOK_URL`, `N8N_WEBHOOK_SECRET`	n8n address and shared secret
`N8N_CORRECTION_WEBHOOK_URL`, `N8N_IMPORT_ERROR_WEBHOOK_URL`	Optional. Derived from the batch URL if empty
`SIMULATE_SCANS`	Fake results, no n8n
`CORS_ORIGINS`, `CORS_ALLOW_LAN`	Which web addresses may call the backend
`ABANDONED_BATCH_HOURS`	Force-close forgotten batches (default 3)
`N8N_TIMEOUT_SECONDS`, `N8N_CORRECTION_TIMEOUT_SECONDS`, `MAX_PIPELINE_ATTEMPTS`	Timeouts (360, 180) and retries (5)
`MAX_UPLOAD_MB`, `INGEST_MAX_CONCURRENCY`, `PREDATE_GRACE_SECONDS`	Upload limits
`WATCHER_ONLINE_SECONDS`, `WATCHER_API_BASE`, `PUBLIC_API_BASE`	Watcher light timing and the address given to watchers
`PAD_FOR_OMR`, `PAD_PERCENT`, `CROP_LONG_BOND`	Image preparation for OMR
`SCAN_IMAGE_BUCKET`, `SCAN_IMAGE_RETENTION_DAYS`, `SCAN_IMAGE_MAX_SIDE`, `SCAN_IMAGE_JPEG_QUALITY`	Page viewing copies
`BREVO_API_KEY` (secret), `BREVO_SENDER_EMAIL`, `BREVO_SENDER_NAME`	Activation emails
`TRUST_PROXY_HEADERS`, `FORWARDED_ALLOW_IPS`	Only when behind a proxy

Frontend settings

- `VITE_API_BASE` (backend address)
- `VITE_SHEET_SERVICE_ACCOUNT_EMAIL` (shown on screen so staff know which email to share a Google Sheet with)

OMR service switches

`OCR_MODE`, `STAMP_FINDER`, `STAMP_READER`, `OMR_MAX_LONG_SIDE`, `OMR_CAP_LONG_SIDE`.

OMR main files

`api.py`, `utlis.py`, `grid_detect.py`, `grid_config.py`, `mark_detect.py`, `stamp_find.py`, `stamp_read.py` (+ weights `stamp_read_weights.pt`, `stamp_templates.npz`), `Dockerfile`, `requirements.txt`.

B Database migrations

Run each in the Supabase SQL Editor, by file name, and read it first. Two files share 004 and two share 007.

FILE	WHAT IT DOES	HOW TO CHECK
`001_batch_scanning`	`stations` and `scan_batches` tables	Tables exist
`002_batch_hardening`	Station key hash, batch counters, `scan_batch_items`, tighter rules. **Required.** Without it the backend returns `schema_outdated`	Table `scan_batch_items`
`003_repair_import_error_scan_images`	One-off repair, optional	n/a
`004_mask_existing_profanity`	One-off clean-up. Back up first, cannot be undone	n/a
`004_watcher_folder_status`	`watcher_*` columns on `stations`	Column on `stations`
`005_scan_review_queue`	Review versioning. Without it review actions return `migration_required`	Column `review_version`
`006_ocr_correction_log`	View `ocr_correction_log`	View exists
`007_audit_queries`	Read-only. Run FIRST	n/a
`007_security_hardening`	Fixes self-signup privilege escalation, RLS	Notes say applied 2026-09-30
`008_scan_unsaved_pages`	Table for "Couldn't be saved" pages	Notes say applied
`009_stations_watcher_skipped_files`	Extra watcher columns	Column `stations.watcher_skipped_files`
`010_scan_page_images`	Page viewer. Needs private bucket `scan-pages`	Bucket exists
`011_instructor_name_matching`	Duplicate-instructor detection	Function `find_duplicate_instructor`
`012_scan_items_sha_index`	Index for the reused-image check	Index on `file_sha256`

FILE	WHAT IT DOES	HOW TO CHECK
`013_sast_null_aware_means`	Blank = NULL; `recompute_sast`	Function `recompute_sast`. Notes (2026-10-08) say live
`014_sast_min_answered`	Fewer than 5 of 20 not counted	Notes (2026-10-16) say applied
`015_stamp_no`	Stamp columns, unique per term, `save_website_raw_row`	Column `stamp_no`
`015b_delete_dummy_scan_data`	**DESTRUCTIVE.** Deletes test scans (backs up to `backup_*` tables). Run only on purpose	n/a
`016_wordcloud_words_backfill`	One-off	n/a
`017_null_placeholder_comments`	Optional clean-up	n/a
`018_sheet_import_jobs`	Table for Google Sheet import jobs	Table exists

C Review flag codes

These are the reasons a form can be sent to review. The full wording is in `frontend\src\messages.js`.

`identity_mismatch` `instructor_missing` `subject_missing` `student_id_missing`
`duplicate_student_id` `reused_image` `orientation_uncertain` `low_confidence`
`multiple_marks` `blank_answers` `grid_not_detected` `omr_unavailable`
`omr_fallback_grid` `omr_agreement_below_30` `layout_mismatch` `stamp_missing`
`stamp_unreadable` `multiple_stamps` `stamp_unconfirmed` `duplicate_stamp`

D Glossary

TERM	MEANING
SAST	The paper evaluation form where students rate an instructor on 20 items
SAO	Student Affairs Office
Batch	One scanning run: one instructor, one subject, one station
Station	A scanning PC registered on the website
Station key	A password for one station, used by the watcher
Watcher	Program on the scanning PC that sends scanned files to the backend
Heartbeat	A "still alive" signal sent every 15 seconds
Backend	The server program that does the work behind the website (FastAPI)
Frontend	The pages you see in the browser (React)
Localhost	The same computer you are using. Not visible on the internet
Webhook	A web address that starts a workflow when something is sent to it
n8n	Tool that runs the step-by-step workflow that reads and saves forms
Workflow / node	The whole flow chart / one box in it
Credential	A saved login or key inside n8n
VPS	A rented server
Docker	Packaging that runs a program in an isolated "container"
Container	One running Docker program (for example omr-api)
OMR	Optical mark recognition: reading ticked boxes
OCR	Optical character recognition: reading text
Gemini	Google's AI model that reads the form text
Supabase	Hosted database, login and file storage
Migration	A SQL file that changes the database structure
RLS	Row-level security: rules for who can read or change each row
Edge function	A small program running inside Supabase
Service key	Supabase secret key with full power. Never expose it
Anon key	Supabase public key for apps. Safe to bundle, limited by RLS
Stamp	Number printed by a numbering machine on each paper form. Unique per term

TERM	MEANING
Flagged	A form sent to review because something looks wrong
Credit	Approve a flagged form so it counts
Placeholder instructor	An instructor record created before the person has a real login
Short / long bond	Paper sizes: 8.5 x 11 and 8.5 x 13 inches
Dpi	Dots per inch: scan sharpness
APK	The Android app install file
Keystore	The file that signs the Android app. Losing it blocks updates
FCM	Firebase Cloud Messaging: push notifications
Brevo	Email sending service used for activation emails
Caddy	Web server on the VPS that adds HTTPS to n8n
DuckDNS	Free service that gives the VPS a web name
Vite	Tool that runs and builds the frontend
pytest	Python test runner
UAT	User acceptance test
.env	A private settings file holding secrets. Never commit it